EOSC 213: Computational Methods in Geological Engineering

Lecture 19: Beyond Euler and Midpoint – a survey of other ODE methods

Shadab Ahamed

Department of Earth, Ocean and Atmospheric Sciences,
The University of British Columbia

Thu, Mar 26, 2026

Today's goal

We already know four core time-stepping methods:

- Forward Euler
- Backward Euler
- Explicit Midpoint
- Implicit Midpoint

Today we will:

1. briefly review these methods,
2. see their geometric meaning,
3. and then survey other important ODE methods.

Main message

Different numerical methods exist because different problems require different combinations of **accuracy, stability, efficiency,** and **special structure preservation**.

Review: Forward Euler

Consider

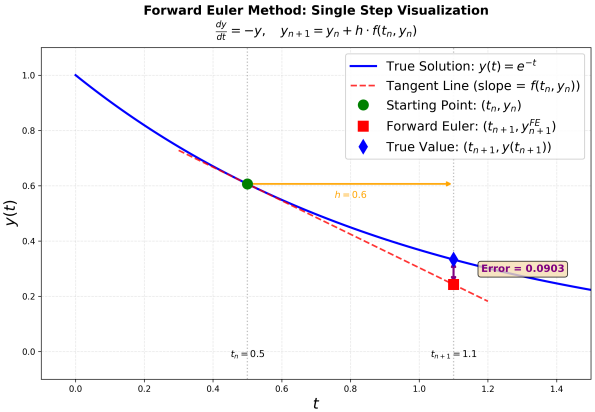
$$\frac{dy}{dt} = f(t, y), \qquad y(t_n) \approx y_n.$$

Forward Euler uses the slope at the **current point**:

$$y_{n+1} = y_n + h\, f(t_n, y_n).$$

Geometric meaning:

- At (t_n, y_n), the ODE gives the slope.
- Draw the tangent line at the current point.
- Move forward by one step h along that tangent line.



Key idea: replace the true curve locally by its tangent line at the start of the step.

Review: Backward Euler

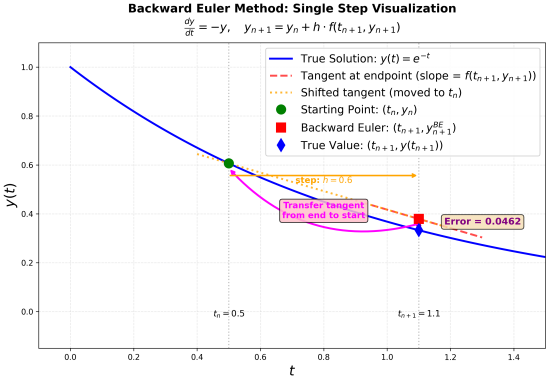
Backward Euler uses the slope at the **next point**:

$$y_{n+1} = y_n + h\,f(t_{n+1}, y_{n+1}).$$

This is an **implicit** method because y_{n+1} appears on both sides.

Geometric meaning:

- We want the tangent line at the **end** of the step.
- The unknown point (t_{n+1}, y_{n+1}) must satisfy the update formula.
- For nonlinear problems, we usually solve this with Newton's method.



Key idea: use future slope information for improved stability, especially for stiff problems.

Review: Explicit Midpoint

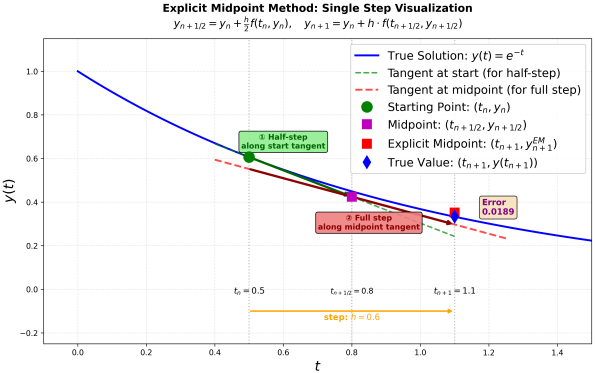
Explicit Midpoint first estimates the solution at the midpoint, then uses the midpoint slope:

$$k_1 = f(t_n, y_n), \quad k_2 = f\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2}k_1\right)$$

$$y_{n+1} = y_n + h\,k_2.$$

Geometric meaning:

- Use the slope at the start to estimate a midpoint.
- Compute a better slope at that midpoint.
- Use that midpoint slope to advance the full step.



Key idea: use more information within the step to improve accuracy.

Review: Implicit Midpoint

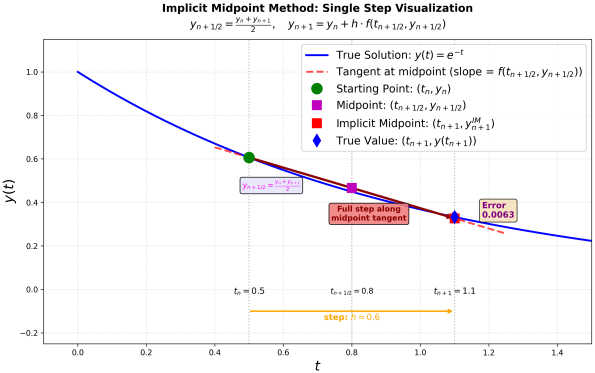
Implicit Midpoint uses the slope at the midpoint, but the midpoint depends on the unknown future value:

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2}\right).$$

This is also an **implicit** method.

Geometric meaning:

- The slope is evaluated at the midpoint in time and solution value.
- But that midpoint includes y_{n+1}, so we must solve for the new value.



Key idea: midpoint information with implicit stability properties.

Why do other ODE methods exist?

The four methods we studied are important, but they are not the whole story.

Different methods were developed because we may want:

- **higher accuracy** without making h extremely small,
- **better stability** for stiff systems,
- **better efficiency** for long simulations,
- **automatic step-size control**,
- or preservation of special structure (for example, in mechanics).

Big picture

There is no single "best" ODE solver for every problem.

Heun's method (Improved Euler)

Heun's method averages two slope estimates:

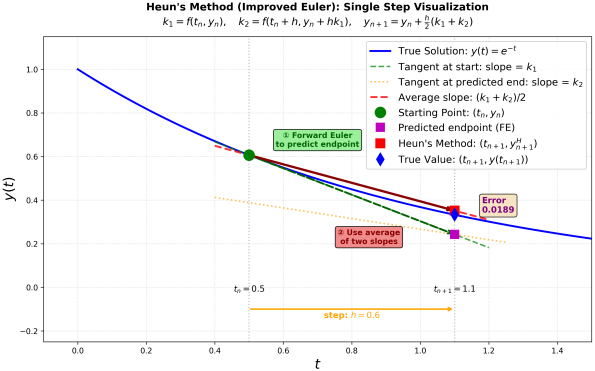
$$k_1 = f(t_n, y_n),$$

$$k_2 = f(t_n + h,\ y_n + hk_1),$$

$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2).$$

Interpretation:

- take one slope at the start,
- one slope at the predicted end,
- then average them.



Why it exists: a simple way to get better accuracy than Forward Euler while staying explicit.

The classical Runge-Kutta Fourth-Order (RK4) method

RK4 uses four slope evaluations within each step:

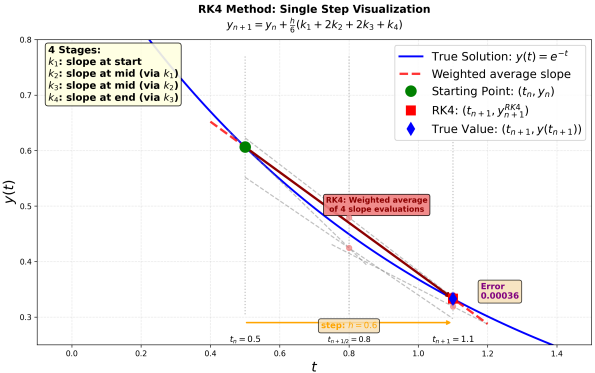
$$k_1 = f(t_n, y_n),$$

$$k_2 = f\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2}k_1\right),$$

$$k_3 = f\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2}k_2\right),$$

$$k_4 = f(t_n + h,\, y_n + hk_3),$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$



Why it exists: much higher accuracy for nonstiff problems, while remaining explicit.

Trapezoidal rule (Crank–Nicolson for ODEs)

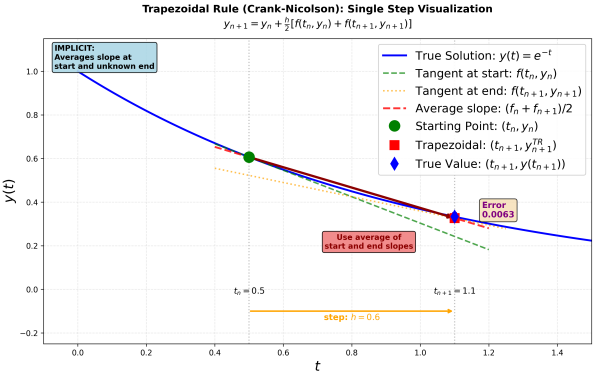
The trapezoidal rule averages the slope at the beginning and end of the step:

$$y_{n+1} = y_n + \frac{h}{2}\Big[f(t_n, y_n) + f(t_{n+1}, y_{n+1})\Big].$$

This is an **implicit** method.

Interpretation:

- use the starting slope,
- use the ending slope,
- average the two.



Why it exists: combines improved accuracy with good stability properties.

Multistep methods

So far, every method we studied is a **one-step method**:

$$y_{n+1} = \text{formula involving } t_n,\ y_n, \text{ and possibly values inside the current step.}$$

Examples:

$$y_{n+1} = y_n + h\,f(t_n, y_n) \qquad \text{(Forward Euler)}$$

$$y_{n+1} = y_n + \frac{h}{2}\Big[f(t_n, y_n) + f(t_{n+1}, y_{n+1})\Big] \qquad \text{(Trapezoidal rule)}$$

A **multistep method** uses information from *multiple previous time levels*:

$$y_{n+1} = \text{formula involving } y_n,\ y_{n-1},\ y_{n-2}, \ldots$$

and/or

$$f(t_n, y_n),\ f(t_{n-1}, y_{n-1}),\ \ldots$$

Main contrast

- **One-step methods:** only need the current value y_n to move forward.
- **Multistep methods:** reuse several previously computed values/slopes.

Examples of multistep methods

A simple explicit multistep method is the **2-step Adams–Bashforth method**:

$$y_{n+1} = y_n + \frac{h}{2}\Big(3f(t_n, y_n) - f(t_{n-1}, y_{n-1})\Big).$$

Notice that this uses information from:

$$(t_n, y_n) \qquad \text{and} \qquad (t_{n-1}, y_{n-1}).$$

A simple **implicit multistep** method is the **2-step Adams–Moulton method**:

$$y_{n+1} = y_n + \frac{h}{12}\Big(5f(t_{n+1}, y_{n+1}) + 8f(t_n, y_n) - f(t_{n-1}, y_{n-1})\Big).$$

- Multistep methods can be more efficient because they reuse old information.
- But they need **starting values** first, usually from a one-step method such as Forward Euler or RK4.
- So in practice: a multistep method often begins with a one-step method.

Other important methods to know by name

- **Adaptive Runge–Kutta methods (RK23, RK45):** automatically change the step size; widely used in software for general nonstiff IVPs.
- **Adams–Bashforth and Adams–Moulton methods:** multistep methods that reuse information from previous steps.
- **Backward Differentiation Formula (BDF) methods:** important implicit multistep methods for stiff systems.
- **Leapfrog / Verlet methods:** commonly used in mechanics because they can preserve physical structure better over long times.
- **Rosenbrock, SDIRK, and IMEX methods:** more advanced methods used when stiffness or mixed fast/slow processes are important.

A simple map of the landscape

Method	Type	Why use it?
Forward Euler	explicit, one-step	simplest possible method
Backward Euler	implicit, one-step	stability; stiff problems
Explicit Midpoint	explicit, one-step	better accuracy than Euler
Implicit Midpoint	implicit, one-step	midpoint accuracy + stability
Heun	explicit, one-step	easy improvement over Euler
RK4	explicit, one-step	very accurate for nonstiff IVPs
Trapezoidal rule	implicit, one-step	accuracy + stability
Adams–Bashforth	explicit, multistep	reuses past slopes efficiently
Adams–Moulton	implicit, multistep	multistep accuracy; used in predictor-corrector ideas
RK45	explicit, adaptive	practical general-purpose solver
BDF	implicit, multistep	stiff systems
Verlet / Leapfrog	structure-preserving	mechanics / long-time motion

Takeaway

What students should remember

- ODE methods come in families.
- Different families are designed for different goals.
- For this course, the important question is not just *how* a method works, but also *when* it should be used.

A useful summary:

- Want a simple method? Use Euler-type ideas.
- Want better accuracy? Use Runge–Kutta methods.
- Want stability for stiffness? Use implicit methods such as Backward Euler, trapezoidal rule, or BDF.
- Want long-time physical fidelity in mechanics? Use structure-preserving methods.